

BIG DATA ANALYSIS LAB

Flight Delay Data Analysis

Using Hadoop MapReduce and Apache Pig

FINAL LAB PROJECT REPORT

Student Name	Nusrat Jahan Ananna
Student ID	0222220005101099
Course	Big Data Analysis Lab
Project Dataset	Flight Delay Dataset - 2024
Submission Date	13-09-2026

Table of Contents

1. Problem Statement
 2. Project Objectives
 3. Dataset Description
 4. System Architecture and Workflow
 5. Hadoop and HDFS Implementation
 6. Java MapReduce Analysis
 7. Apache Pig Analysis
 8. HDFS Browser Verification
 9. MapReduce vs Apache Pig
 10. Key Findings
 11. Limitations and Future Improvements
 12. Conclusion
 13. GitHub Repository
- Appendix A. Verified Project Structure

1. Problem Statement

Commercial aviation generates millions of operational records containing scheduled and actual departure/arrival times, delays, cancellations, routes, carriers and airport information. Processing a full-year dataset of this scale with ordinary single-machine techniques can be slow and inconvenient. This project applies the Hadoop ecosystem to store and analyze the 2024 Flight Delay dataset using HDFS, Java MapReduce and Apache Pig.

The project focuses on practical questions such as airline traffic volume, departure delay, arrival delay by origin airport, origin-airport traffic, cancellations by airline and monthly flight volume. The implementation also verifies that MapReduce output files are accessible through the Hadoop NameNode web interface.

2. Project Objectives

- Store a large real-world flight dataset in Hadoop Distributed File System (HDFS).
- Process 7+ million flight records with Java MapReduce.
- Analyze airline, airport, delay and cancellation patterns.
- Use Apache Pig for higher-level aggregation and compare its workflow with MapReduce.
- Verify HDFS directories and output files through both CMD and the Hadoop browser interface.
- Maintain a reproducible project structure suitable for GitHub submission.

3. Dataset Description

The project uses the Flight Delay Dataset - 2024. The CSV file is approximately 1.31 GB and contains 7,079,081 data records plus one header row, with 35 columns. The attributes include date, airline/carrier, flight number, origin, destination, scheduled and actual times, departure and arrival delays, cancellation/diversion status, elapsed time, distance and delay-cause fields.

Item	Value
Dataset	Flight Delay Dataset - 2024
Main file	flight_data_2024.csv
Approx. size	1.31 GB (shown as 1.2 G in HDFS)
Data records	7,079,081
Columns	35
Format	CSV
HDFS path	/flight_delay_project/input/flight_data_2024.csv

Because some text fields contain commas inside quotation marks (for example city names), the Java programs use CSV-aware parsing rather than a simple comma split. This prevents quoted fields from shifting column positions.

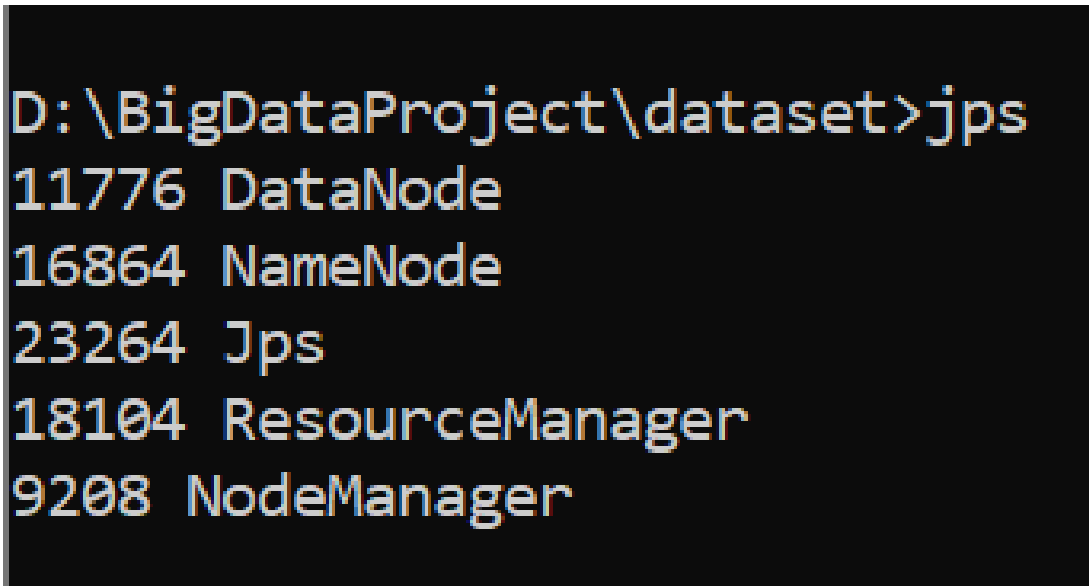
4. System Architecture and Workflow

The project follows a simple big-data pipeline: local CSV dataset -> HDFS input -> Java MapReduce / Apache Pig processing -> HDFS output -> CMD and browser verification -> result interpretation and GitHub documentation.

Layer	Technology / Role
Storage	Hadoop HDFS
Batch processing	Java MapReduce
High-level analysis	Apache Pig
Execution environment	Windows CMD, Java 8, Hadoop 3.2.4
Verification	HDFS CLI and NameNode browser (localhost:9870)
Version control	Git / GitHub

5. Hadoop and HDFS Implementation

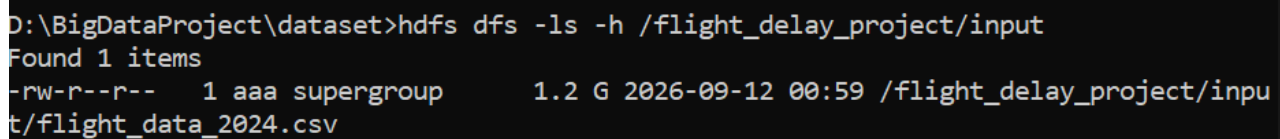
The Hadoop services were started and verified using the Java Process Status command. The screenshot below confirms that NameNode, DataNode, ResourceManager and NodeManager were running during the project.



```
D:\BigDataProject\dataset>jps
11776 DataNode
16864 NameNode
23264 Jps
18104 ResourceManager
9208 NodeManager
```

Figure 1. Verification of Hadoop services using the jps command.

The project dataset was then stored under the HDFS input directory. The HDFS listing reports the file as approximately 1.2 G, confirming that the large CSV was successfully uploaded.



```
D:\BigDataProject\dataset>hdfs dfs -ls -h /flight_delay_project/input
Found 1 items
-rw-r--r--  1 aaa supergroup      1.2 G 2026-09-12 00:59 /flight_delay_project/input/flight_data_2024.csv
```

Figure 2. Flight Delay 2024 dataset stored in the HDFS input directory.

A dedicated HDFS project hierarchy was used to keep the input data and generated outputs organized.

```
D:\BigDataProject\dataset>hdfs dfs -ls /flight_delay_project
Found 3 items
drwxr-xr-x  - aaa supergroup          0 2026-09-12 00:59 /flight_delay_project/input
drwxr-xr-x  - aaa supergroup          0 2026-09-12 23:05 /flight_delay_project/mapreduce_output
drwxr-xr-x  - aaa supergroup          0 2026-09-12 01:15 /flight_delay_project/pig_output

D:\BigDataProject\dataset>_
```

Figure 3. HDFS project directory structure containing input, mapreduce_output and pig_output.

6. Java MapReduce Analysis

Five MapReduce analyses were completed. Existing HDFS output files were read with `hdfs dfs -cat` for verification; therefore the long-running jobs did not need to be executed again merely to capture report screenshots.

6.1 Total Flights by Airline

This job groups all valid flight records by operating carrier and counts the number of flights for each airline. The counts sum to all 7,079,081 data records. WN has the largest volume with 1,419,419 flights, followed by DL with 1,009,194 and AA with 984,306.

```
D:\BigDataProject\dataset>hdfs dfs -cat /flight_delay_project/mapreduce_output/flights_by_airline/part-r-00000
9E      200094
AA      984306
AS      245819
B6      240282
DL      1009194
F9      208624
G4      117210
HA      78530
MQ      279955
NK      261103
OH      227971
OO      744658
UA      760451
WN      1419419
YX      301465
D:\BigDataProject\dataset>
```

Figure 4. MapReduce output - total flights by airline.

Carrier	Flights
WN	1,419,419
DL	1,009,194
AA	984,306
UA	760,451
OO	744,658
HA	78,530

6.2 Average Departure Delay by Airline

The second MapReduce program computes the mean departure delay for each airline using records with available departure-delay values. AA records the highest average among the listed carriers at approximately 20.70 minutes, while YX has the lowest at approximately 3.97 minutes.

```
D:\BigDataProject\dataset>hdfs dfs -cat /flight_delay_project/mapreduce_output/average_departure_delay/part-r-00000
9E      8.336983907153115
AA      20.698094838335074
AS      7.634070904696161
B6      17.458447067970354
DL      10.22354855995713
F9      19.002299053417453
G4      14.047917444171535
HA      5.468065400002573
MQ      10.176118484184606
NK      14.957090069103835
OH      13.93673215662334
OO      11.279936984775848
UA      11.95902404872007
WN      11.748602732954195
YX      3.9667591711062045
```

Figure 5. MapReduce output - average departure delay by airline.

6.3 Average Arrival Delay by Origin Airport

This analysis groups valid arrival-delay observations by origin airport. A total of 348 origin-airport groups were produced. Positive values indicate average late arrival, while negative values indicate that flights associated with that origin arrived early on average.

```
D:\BigDataProject\dataset>hdfs dfs -cat /flight_delay_project/mapreduce_output/average_arrival_delay_by_origin/part-r-00000
ABE 3.2024958794443137
ABI 4.409534750143596
ABQ 4.291413230420739
ABR 10.718836565096954
ABY 15.914134742404228
ACK 11.23023402909551
ACT 15.302830188679245
ACV 17.972957084068195
ACY 1.755950385517935
ADK -6.833333333333333
ADQ 1.3479349186483105
AEX 12.220093215950286
AGS 13.461440570119624
AKN -3.2655367231638417
ALB 6.072989621129019
ALW 2.8155893536121672
AMA 5.493410543130991
ANC 1.6699247707429576
APN 9.789819376026273
ASE 20.62104830983988
ATL 6.82433913203144
ATW 10.712748614281056
AUS 9.236286419284426
AVL 11.659344431096619
AVP 13.363520408163266
AZA -0.957611120645272
AZO 5.521739130434782
BDL 5.913514991342386
BET 12.032033426183844
BFF 9.162295081967214
BFL 1.2623445156426687
BGM 7.265536723163842
BGR 3.998432601880878
BHM 7.251707891637221
BIH 9.047619047619047
BIL 3.3932310689067444
BIS 5.029546612328069
BJI 9.915629322268327
BLI 3.810071371927042
BLV 15.130393996247655
BMI 4.794991263832266
BNA 5.92740791766279
BOI 2.9826324605483556
BOS 4.108080707824065
BPT 4.1678321678321675
BQK 17.442622950819672
BQN 10.617402164862614
AMA 5.493410543130991
```

Figure 6. MapReduce output - average arrival delay by origin airport (partial output).

Examples visible in the output include ABE = 3.20 minutes, ABQ = 4.29 minutes, ABY = 15.91 minutes and ADK = -6.83 minutes. The negative ADK value illustrates that average delay measures can also represent early arrival.

6.4 Total Flights by Origin Airport

The fourth job counts flights by origin airport. This provides a direct view of airport traffic volume in the dataset. For example, the displayed results include ATL with 341,910 flights, AUS with 85,344 and ABQ with 24,092.

```

D:\BigDataProject\dataset>hdfs dfs -cat /flight_delay_project/mapreduce_output/flights_by_origin_airpor
t/part-r-00000
ABE      4337
ABI      1757
ABQ     24092
ABR       728
ABY       765
ACK     1675
ACT     1087
ACV     1737
ACY     3052
ADK       104
ADQ       816
AEX     1966
AGS     4022
AKN       178
ALB    12042
ALW       544
AMA     5086
ANC    18543
APN       629
ASE     7888
ATL   341910
ATW     6259
AUS    85344
AVL    12082
AVP     1592
AZA     5884
AZO     1005
BDL    22302
BET       726

```

Figure 7. MapReduce output - total flights by origin airport (partial output).

6.5 Cancelled Flights by Airline

The cancellation analysis counts records marked as cancelled for each operating carrier. AA has the largest cancellation count in the result with 15,252, followed by UA with 12,478 and WN with 11,772.

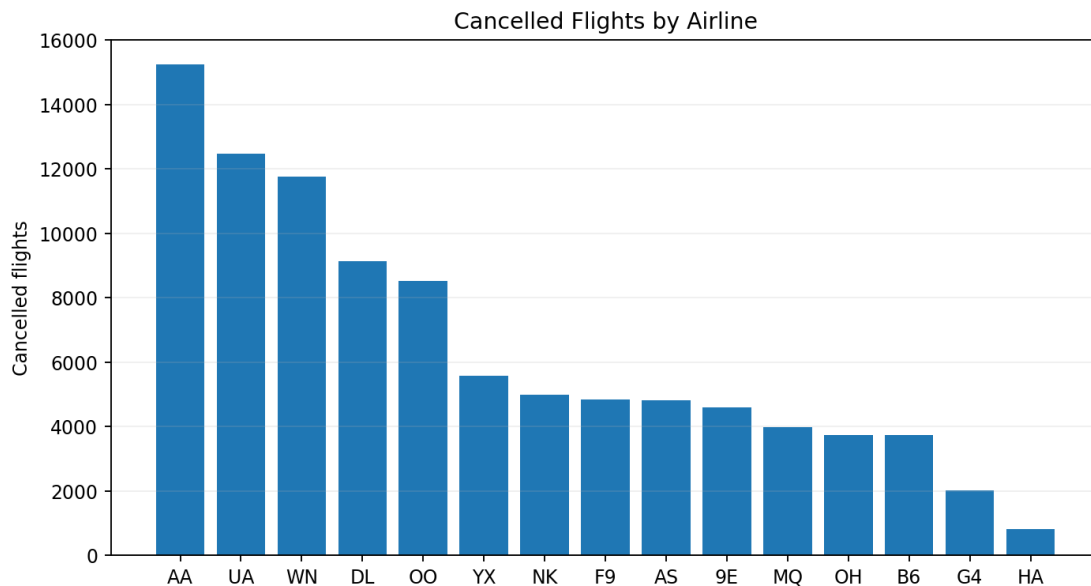


Figure 8. Cancelled flights by airline - project result visualization.

Carrier	Cancelled Flights
AA	15,252
UA	12,478

WN	11,772
DL	9,147
OO	8,527
YX	5,576

7. Apache Pig Analysis

Apache Pig was used as a higher-level data-processing approach. Two analyses were completed: total flights by airline and total flights by month. The Pig scripts use early CSV fields required for these aggregations. The screenshots below show saved outputs from the completed Pig executions, displayed from local text files so that the lengthy Pig jobs did not have to be rerun only for documentation.

7.1 Total Flights by Airline

The Pig airline aggregation produced the same carrier counts as the Java MapReduce analysis, providing a useful consistency check. The completed Pig execution took approximately 27 minutes 10.659 seconds.

```
D:\BigDataProject\pig>type D:\BigDataProject\output\pig_flights_by_airline.txt
(WN,1419419)
(DL,1009194)
(AA,984306)
(UA,760451)
(OO,744658)
(YX,301465)
(MQ,279955)
(NK,261103)
(AS,245819)
(B6,240282)
(OH,227971)
(F9,208624)
(9E,200094)
(G4,117210)
(HA,78530)
```

Figure 9. Saved output from the completed Apache Pig analysis - total flights by airline.

7.2 Total Flights by Month

The monthly analysis shows how flight volume changes across 2024. July has the highest volume with 634,613 flights, while February has the lowest with 519,221. The monthly counts sum to the complete 7,079,081-record dataset. The completed Pig execution took approximately 26 minutes 48.177 seconds.

```
D:\BigDataProject\pig>type D:\BigDataProject\output\pig_flights_by_month.txt
(1,547271)
(10,615497)
(11,575404)
(12,590581)
(2,519221)
(3,591767)
(4,582205)
(5,609743)
(6,611132)
(7,634613)
(8,619025)
(9,582622)
```

Figure 10. Saved output from the completed Apache Pig analysis - total flights by month.

Month	Flights
January	547,271

February	519,221
March	591,767
April	582,205
May	609,743
June	611,132
July	634,613
August	619,025
September	582,622
October	615,497
November	575,404
December	590,581

8. HDFS Browser Verification

In addition to command-line verification, the Hadoop NameNode web interface was used to browse the MapReduce output directory. This provides visual confirmation that all five analysis directories exist in HDFS.

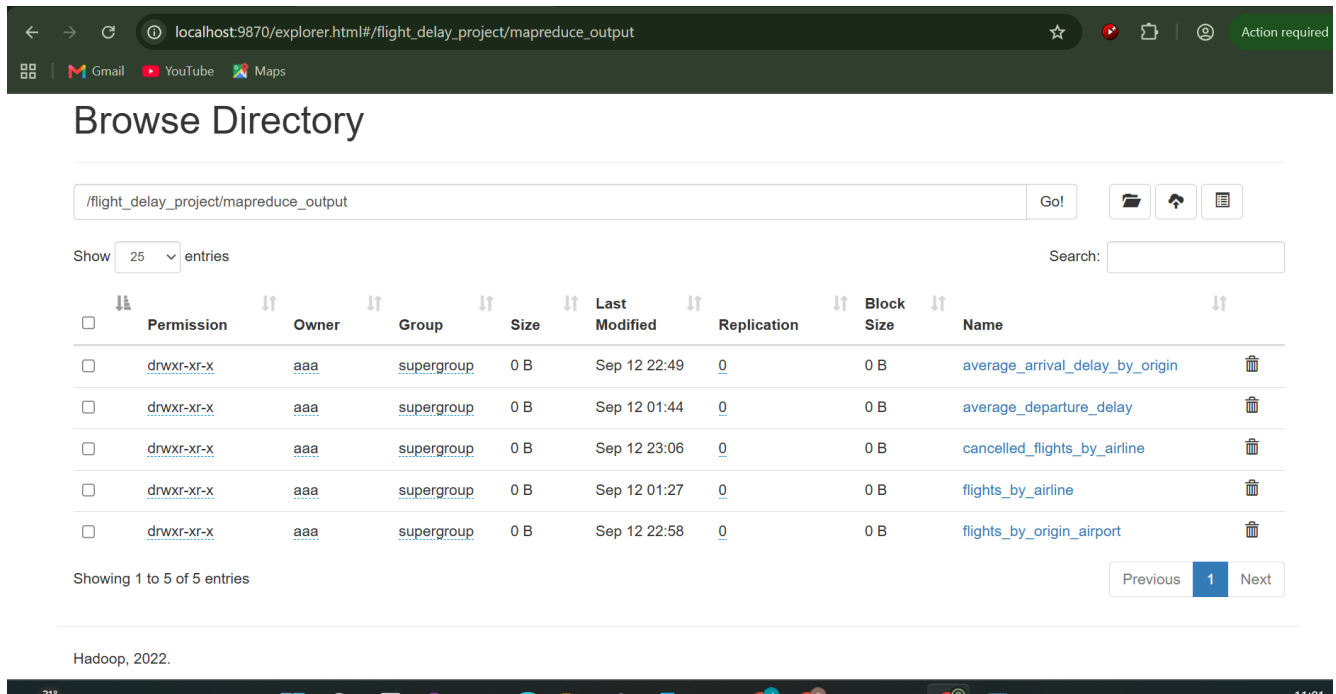


Figure 11. Hadoop HDFS web interface showing the five MapReduce output directories.

The `flights_by_airline` output file was opened from the browser. The file-information view displays block metadata and also exposes the file contents, confirming that `part-r-00000` is stored and readable in HDFS.

File information - part-r-00000

[Download](#)[Head the file \(first 32K\)](#)[Tail the file \(last 32K\)](#)

Block information -- Block 0 ▾

Block ID: 1073741846

Block Pool ID: BP-1263339085-192.168.0.103-1788030542563

Generation Stamp: 1022

Size: 151

Availability:

- DESKTOP-JRK564J

File contents

```
9E 200094
AA 984306
AS 245819
B6 240282
DL 1009194
```

Figure 12. HDFS file information and browser verification of `flights_by_airline/part-r-00000`.

The browser output begins with the same airline counts obtained through the HDFS command line, such as 9E = 200,094, AA = 984,306, AS = 245,819, B6 = 240,282 and DL = 1,009,194. This cross-check supports the correctness of the stored MapReduce result.

9. MapReduce vs Apache Pig

Aspect	Java MapReduce	Apache Pig
Programming level	Low-level Java implementation	Higher-level data-flow scripting
Development effort	More code: Mapper, Reducer, Driver	Shorter scripts using LOAD, GROUP, FOREACH
Control	Fine-grained control over parsing and processing	Convenient for rapid analytical queries
Output validation	HDFS part-r-00000 files	Pig DUMP / saved result text
Project use	Five core analyses	Two complementary analyses
Observed airline result	7,079,081 flights total	Same airline counts as MapReduce

MapReduce is appropriate when precise control over parsing and processing is required, especially for a quoted CSV dataset. Pig reduces implementation complexity for aggregation-style analysis. In this project the matching flights-by-airline results demonstrate that both approaches can produce consistent results when the required fields are parsed correctly.

10. Key Findings

- The dataset contains 7,079,081 flight records across 35 attributes and is approximately 1.31 GB.
- WN is the largest carrier by flight count with 1,419,419 records.
- Among the airline averages produced by the project, AA has the highest average departure delay at about 20.70 minutes and YX the lowest at about 3.97 minutes.
- ATL appears as a major origin in the displayed airport-count output with 341,910 flights.
- AA has the highest cancellation count at 15,252, followed by UA (12,478) and WN (11,772).
- July is the busiest month with 634,613 flights; February is the lowest with 519,221.
- Pig and MapReduce produce matching airline flight counts, providing an internal validation of the aggregation.

11. Limitations and Future Improvements

The project is a batch-processing analysis and does not model live flight streams. Some delay fields contain missing values, so average-delay analyses necessarily use fewer records than the full dataset. The current work also focuses on descriptive aggregation rather than predictive modeling.

Future work could add Hive or Spark SQL, visual dashboards, delay-cause analysis, route-level analysis, seasonal comparisons, cancellation rates rather than raw counts, and machine-learning models for delay prediction. A quote-aware CSV loader or pre-cleaned structured format such as Parquet would also simplify downstream analytics.

12. Conclusion

This project successfully demonstrates an end-to-end big-data workflow using a real 2024 flight dataset containing more than seven million records. The data was stored in HDFS, processed with five Java MapReduce programs and two Apache Pig analyses, and verified through both command-line output and the Hadoop web interface. The results reveal clear patterns in airline volume, airport traffic, delays, cancellations and monthly flight activity.

The work also demonstrates the practical difference between lower-level MapReduce programming and higher-level Pig scripting. MapReduce provides strong control for CSV-aware processing, while Pig offers a concise way to express aggregation tasks. Together they form a complete lab project that combines distributed storage, batch processing, result validation and reproducible project organization.

13. GitHub Repository

The source code, scripts, outputs and project screenshots are organized in the GitHub repository named Flight-Delay-Big-Data-Analysis. The large raw CSV dataset is intentionally excluded from Git version control because of its size; the dataset source and project instructions should be documented in the repository README.

Repository: <https://github.com/NUSRATANANYA/Flight-Delay-Big-Data-Analysis>

Appendix A. Verified Project Structure

Local project root: D:\BigDataProject

HDFS project root: /flight_delay_project

HDFS Location	Purpose
/flight_delay_project/input	Input CSV dataset
/flight_delay_project/mapreduce_output/flights_by_airline	Airline flight counts
/flight_delay_project/mapreduce_output/average_departure_delay	Average departure delay
/flight_delay_project/mapreduce_output/average_arrival_delay_by_origin	Average arrival delay by origin
/flight_delay_project/mapreduce_output/flights_by_origin_airport	Origin-airport flight counts
/flight_delay_project/mapreduce_output/cancelled_flights_by_airline	Cancellation counts
/flight_delay_project/pig_output	Reserved Pig output directory